POSTS AND TELECOMMUNICATIONS INSTITUTE
OF TECHNOLOGY
FACULTY OF INFORMATION TECHNOLOGY 1

SOFTWARE ARCHITECTURE AND DESIGN

ESSAY V1

Building an E-Commerce System Using Microservices and AI

Lecturer: **Tran Dinh Que**

Student: **B22DCCN441 – Nguyen Duc Khai**

Class: **E22CNPM01**

Hanoi - 2026

Contents

List of Figures

List of Tables

1 Introduction

Electronic commerce is a representative domain for studying software architecture because the business problem is naturally split across identity, catalog, cart, order, payment, shipment, and customer support capabilities. A small prototype can be implemented as a monolithic Django application, but a system that must support independent evolution, independent scaling, and AI-assisted shopping is better analyzed through the lens of microservices and Domain Driven Design (DDD). This report studies such a system from the perspective of software architecture and design. The implemented project is a Docker-based Django commerce demo with six application services, an Nginx gateway, MySQL and PostgreSQL databases, and an AI chatbot service supported by file-based retrieval artifacts and an optional Neo4j behavior graph.

The purpose of the report is not only to restate the theory of microservices, but to connect architectural concepts to concrete source code. The current implementation keeps `user_service` as the owner of customer and staff web user interfaces, Django session authentication, JWT API authentication, gateway metadata, editorial content, and chatbot proxying. `product_service` owns the unified catalog API with ten categories and one hundred seeded products. `order_service` owns cart, saved items, compare items, orders, checkout orchestration, and analytics. `payment_service` and `shipping_service` own their respective internal lifecycles. Finally, `chatbot_service` owns chat replies, behavior ingestion, product recommendation logic, RAG artifacts, model artifacts, and optional Neo4j graph retrieval.

1.1 Objectives and Scope

This essay has four objectives. First, it explains the architectural transition from a monolithic design to a microservice design and shows why DDD is useful for service decomposition. Second, it documents the e-commerce system requirements and maps them to bounded contexts. Third, it presents the implemented system architecture, data ownership model, API gateway, security approach, and deployment design. Fourth, it analyzes the AI product consultation service, including behavior events, model selection, retrieval-augmented generation, Neo4j graph context, and customer-facing integration points.

The scope is Version 1 of the academic essay. The report focuses on design, architecture, implementation evidence, and evaluation. It does not claim production readiness. The current project is an educational commerce system designed for demonstrating microservice boundaries, service-to-service communication, and AI-assisted customer experience.

1.2 Methodology

The analysis is based on three sources of evidence: the course reference document for the essay structure, the project repository, and the generated evidence artifacts in the project. Source code files were inspected for models, service clients, views, management commands, Docker configuration, Nginx routing, and AI pipeline behavior. The resulting essay therefore describes the system as it is implemented rather than as an abstract template.

2 From Monolithic Architecture to Microservices and DDD

2.1 Monolithic Architecture

A monolithic architecture deploys the user interface, business logic, and data access layer as one application unit. In an e-commerce example, product browsing, user management, cart logic, checkout, payment, delivery status, and recommendation logic could all be located inside a single Django project and backed by a single database. This approach has practical advantages during the early phase of a project: the development environment is simple, deployment is direct, and transactions across modules are easier because all data is local to one runtime.

However, monolithic architecture becomes more difficult to maintain as the system grows. A change to the payment flow may require redeploying the entire application. A scaling need in catalog reads may force the whole application to scale, even if checkout traffic is low. A bug in one module may affect unrelated features because the runtime boundary is shared. In team-based development, multiple developers may also compete inside the same codebase and database schema. These problems are architectural rather than only technical; they are caused by weak boundaries.

2.2 Microservices Architecture

Microservices architecture decomposes the system into small, independently deployable services. Each service encapsulates a business capability, owns its data, and communicates with other services through explicit APIs. The project analyzed in this essay applies this principle by separating the system into `user_service`, `product_service`, `order_service`, `payment_service`, `shipping_service`, and `chatbot_service`. An Nginx gateway exposes the main public entry point on port 8080 and routes traffic to service hostnames inside the Docker network.

The key benefit of this design is independent change. Catalog seed data and product APIs can evolve without changing the payment service. Payment and shipping lifecycles can be tested and deployed as internal services without exposing their databases to the UI. The chatbot service can build RAG artifacts, train behavior models, and query Neo4j without pushing AI dependencies into the order service. This supports the microservice principles of high cohesion, loose coupling, and fault isolation.

The tradeoff is distributed complexity. A checkout request now crosses service boundaries. If payment creation succeeds but shipment creation fails, the order service must handle compensation. Network calls require timeouts, internal keys, error responses, and stable data contracts. These costs are acceptable only if the service boundaries match real business boundaries.

2.3 Domain Driven Design

Domain Driven Design is useful because it encourages the team to model software around business concepts rather than technical layers. In DDD, a bounded context defines a boundary inside which a domain model has a specific meaning. In this project, the concept of a "product" means a canonical catalog entry inside `product_service`, but it becomes a snapshot inside `order_service`. This distinction is important. The

order service does not own the product catalog; it stores product name, brand, category, image, unit price, and quantity as historical order data so that old orders remain readable even if the catalog later changes.

DDD also clarifies integration relationships. The UI context consumes the catalog context through an Open Host Service style API. The commerce context acts as a customer of payment and shipping contexts. The AI assistant context consumes catalog data and behavior data to produce recommendations, but it does not own checkout state. These boundaries prevent an AI feature from becoming a hidden dependency inside every service.

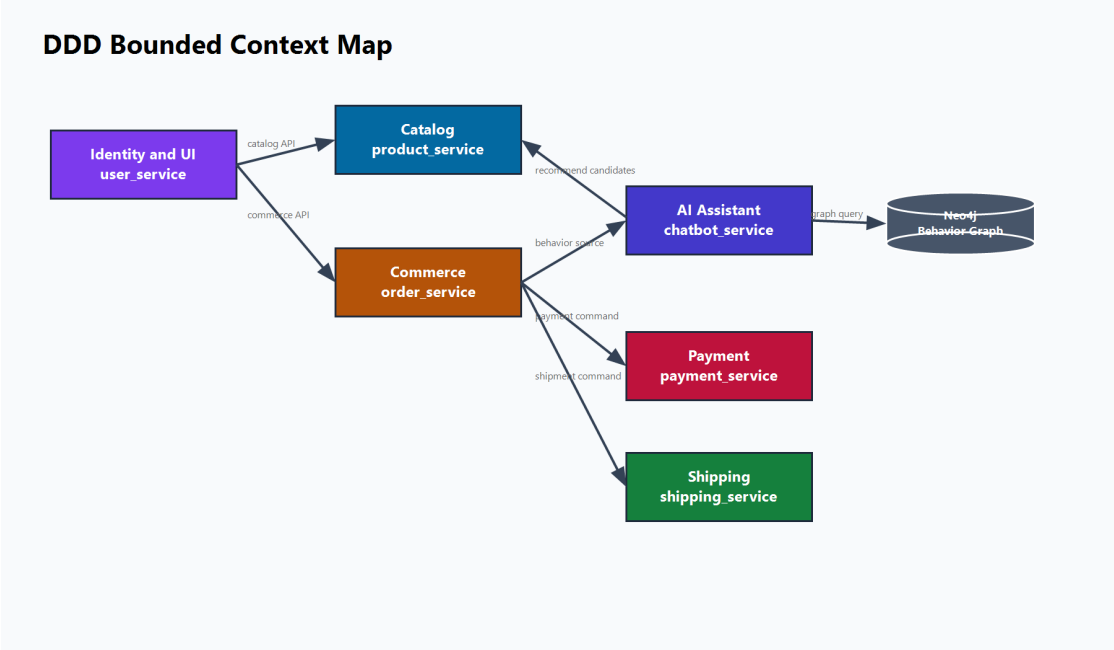


Figure 1: DDD bounded context map for the implemented commerce system.

3 Requirements and Domain Decomposition

3.1 Functional Requirements

The implemented system supports the following functional requirements:

1. Customer registration, login, dashboard browsing, cart management, checkout, order listing, and payment confirmation.

2. Staff registration, login, dashboard access, product management, customer inspection, order inspection, and shipping status updates.

3. A unified product catalog with categories, product search, filtering, brand data, price, stock, and images.

4. Cart, saved list, comparison list, checkout, order snapshots, payment snapshots, shipping snapshots, and customer analytics.

5. Internal payment record creation, confirmation, cancellation rules, and status tracking.

6. Internal shipment record creation, shipping status transitions, and staff-managed delivery status updates.

7. AI-assisted shopping through dashboard suggestions, cart buy-together suggestions, and a floating chatbot widget.

8. Behavior event persistence and recovery through a backfill command in `user_service`.

3.2 Non-functional Requirements

The design also addresses non-functional requirements:

- **Scalability**: catalog, order, payment, shipping, and AI services can be scaled independently in principle.
- **Maintainability**: each service owns a limited domain model and a service-local database.
- **Security**: browser UI uses Django session authentication, API authentication uses JWT, and internal APIs use service keys such as `X-Internal-Key`.
- **Availability**: the chatbot can fall back to file-based RAG and rule-based responses when Neo4j or an LLM provider is unavailable.
- **Deployability**: Docker Compose defines repeatable runtime wiring for databases, services, volumes, and the Nginx gateway.
- **Compatibility**: customer and staff routes remain stable while service boundaries are introduced behind the UI.

3.3 Bounded Contexts

Table 1: Domain decomposition into bounded contexts.

Bounded context	**Service**	**Main responsibility**
Identity and UI	`user_service`	Customer/staff UI, session authentication, JWT API auth, user profile payloads, editorial content, chatbot proxy, gateway inspection page.
Catalog	`product_service`	Category and product catalog, seeded catalog data, catalog search and filtering, staff-protected writes.
Commerce	`order_service`	Cart, saved items, compare items, checkout orchestration, order and item snapshots, analytics, behavior-source export.
Payment	`payment_service`	Payment record lifecycle, confirmation rules, cancellation rules, payment status persistence.

Bounded context	Service	Main responsibility
Shipping	`shipping_service`	Shipment address and status lifecycle, valid transition rules, delivery status updates.
AI assistant	`chatbot_service`	Chat reply API, behavior event storage, model artifacts, RAG knowledge base, optional Neo4j graph retrieval.

4 System Architecture

4.1 Overall Architecture

The implemented system follows a database-per-service microservice style. The Nginx gateway is the primary public entry point at `http://localhost:8080/`. Browser routes such as `/customer/`, `/staff/`, `/admin/`, and `/gateway/` are routed to `user_service`. Public catalog API routes are routed to `product_service`. Order, payment, shipping, and chat APIs are also routed by Nginx, while the order, payment, and shipping services remain internal-first from the application perspective.

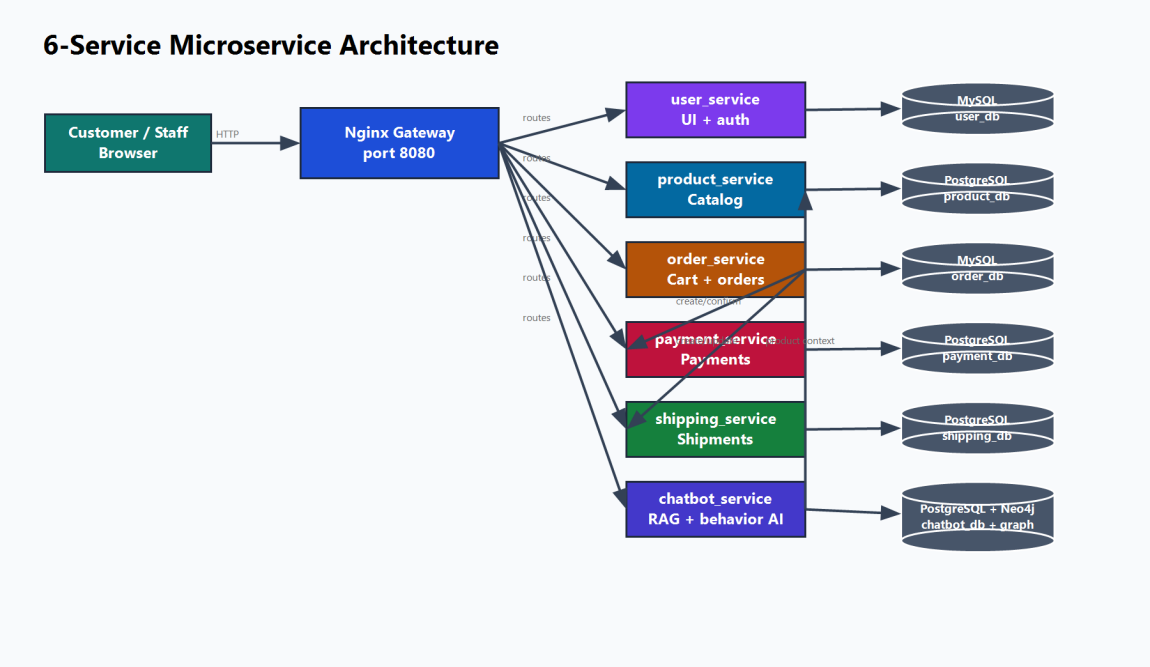


Figure 2: Implemented Docker Compose architecture with Nginx gateway and six application services.

4.2 Service Matrix

Table 2: Runtime service matrix.

Service	Role	Database	Port
gateway	Nginx public entry point and reverse proxy	None	8080
user_service	UI, session auth, JWT auth, staff/-customer flows	MySQL user_db	8000, 8003
product_service	Unified catalog API	PostgreSQL product_db	8001
order_service	Cart, compare, saved, checkout, orders	MySQL order_db	internal
payment_service	Payment records and confirmation	PostgreSQL payment_db	internal
shipping_service	Shipment records and status transitions	PostgreSQL shipping_db	internal
chatbot_service	Chat, RAG, behavior events, graph context	PostgreSQL chatbot_db, Neo4j, files	8005

4.3 API Gateway Design

The Nginx gateway provides a stable entry point and hides internal service hostnames from browser users. Its routing rules map UI routes and authentication APIs to `user-service`, product APIs to `product-service`, cart and order APIs to `order-service`, payment APIs to `payment-service`, shipment APIs to `shipping-service`, and chat APIs to `chatbot-service`. The gateway does not replace service-level authorization; it routes traffic, while the services still enforce JWT, session, staff key, or internal key checks.

Listing 1: Simplified Nginx gateway routing concept.

```
location /customer/      { proxy_pass http://user-service:8000; }
location /staff/         { proxy_pass http://user-service:8000; }
location /api/products/  { proxy_pass http://product-service:8000; }
location /api/cart/      { proxy_pass http://order-service:8000; }
location /api/orders/    { proxy_pass http://order-service:8000; }
location /api/payments/  { proxy_pass http://payment-service:8000; }
location /api/shipments/ { proxy_pass http://shipping-service:8000; }
location /api/chat/      { proxy_pass http://chatbot-service:8000; }
```

5 Service Design

5.1 User Service

`user_service` is the integration-facing service for customer and staff browser workflows. It uses Django's authentication system and keeps browser authentication separate

from API authentication. Customer and staff UI sessions use separate cookie names, which allows both roles to be demonstrated in the same browser. API authentication is exposed through endpoints such as `/api/auth/register/`, `/api/auth/token/`, `/api/auth/token/refresh/`, and `/api/auth/me/`.

The service also acts as a facade over the internal commerce services. For example, when a customer opens the cart page, `user_service` calls `order_service`. When the dashboard needs products, it calls `product_service`. When the customer uses the chatbot widget, it proxies the request to `chatbot_service`. This design preserves stable browser routes while allowing backend service decomposition.

Important data models in this context include `BlogPost`, `Testimonial`, and `LegacyUserMapping`. The use of `LegacyUserMapping` indicates that the architecture was designed with migration in mind, not only with a greenfield prototype assumption.

5.2 Product Service

`product_service` owns the catalog. Its model is intentionally unified rather than split into separate laptop, mobile, and accessory services. The `Category` model defines category name, slug, description, hero image URL, sort order, and active state. The `Product` model belongs to a category and stores name, brand, description, image URL, price, stock, and timestamps. The seed data defines ten categories and one hundred products, including business laptops, gaming laptops, ultrabooks, smartphones, tablets, smartwatches, audio, keyboards and mice, chargers and cables, and bags and stands.

Listing 2: Core catalog model structure.

```
class Category(models.Model):
    name = models.CharField(max_length=120, unique=True)
    slug = models.SlugField(max_length=120, unique=True)
    description = models.TextField(blank=True)
    hero_image_url = models.URLField(blank=True)
    sort_order = models.PositiveIntegerField(default=0)
    is_active = models.BooleanField(default=True)

class Product(models.Model):
    category = models.ForeignKey(Category, on_delete=models.PROTECT,
                                 related_name="products")
    name = models.CharField(max_length=255)
    brand = models.CharField(max_length=120, default="Generic")
    description = models.TextField(blank=True)
    image_url = models.URLField(blank=True)
    price = models.DecimalField(max_digits=12, decimal_places=2)
    stock = models.PositiveIntegerField(default=0)
```

This model supports catalog API endpoints such as `GET /api/categories/`, `GET /api/products/`, and `GET /api/products/<id>/`. Staff write operations are protected with `X-Staff-Key`, which keeps catalog mutation separate from public browsing.

5.3 Order Service

`order_service` owns commerce state: cart items, saved items, compare items, orders, order items, and shipping snapshots. It also orchestrates checkout. The design uses a

`ProductSnapshotMixin` for cart, saved, compare, and order item records. This is an important architectural choice because order history should remain stable even when catalog product data changes.

The checkout flow uses a database transaction to create an order, create shipping details, bulk-create order items, clear the cart, request payment creation from `payment_service`, and request shipment creation from `shipping_service`. If shipment creation fails after payment creation, the code attempts to cancel the created payment. This is a simple compensation pattern suitable for an educational synchronous workflow.

5.4 Payment Service

`payment_service` owns payment persistence and payment state transitions. Its model stores `order_id`, `user_id`, `amount`, `status`, and `paid_at`. The status values are `pending`, `paid`, `failed`, and `cancelled`. A payment can be confirmed or cancelled only while it is pending. This keeps payment invariants close to the payment data model instead of scattering them throughout the UI.

5.5 Shipping Service

`shipping_service` owns shipment address data and shipping status. Its transition map allows movement from `pending` to `preparing`, `shipped`, `delivered`, or `cancelled`; from `preparing` to `shipped`, `delivered`, or `cancelled`; from `shipped` to `delivered`; and prevents invalid movement from terminal states. The order service mirrors a snapshot of the current shipping status for existing UI compatibility, but the shipment service owns the lifecycle record.

5.6 Chatbot Service

`chatbot_service` is the AI assistant context. It exposes a chat reply endpoint, records behavior events, predicts user category affinity, retrieves product and FAQ context, optionally queries Neo4j, and then calls a configured LLM provider. If the provider is unavailable, the service returns a catalog-based fallback response. This design is important because the customer experience remains usable even when external AI dependencies fail.

6 Data and Persistence Design

6.1 Database-per-Service

The system follows the database-per-service principle. User and order data are stored in MySQL, while product, payment, shipping, and chatbot behavior data are stored in PostgreSQL. Neo4j is used as an optional graph database for behavior-context queries. File artifacts under `services/chatbot_service/chatbot/artifacts` store the RAG knowledge base, training data, model artifacts, metrics, and runtime configuration.

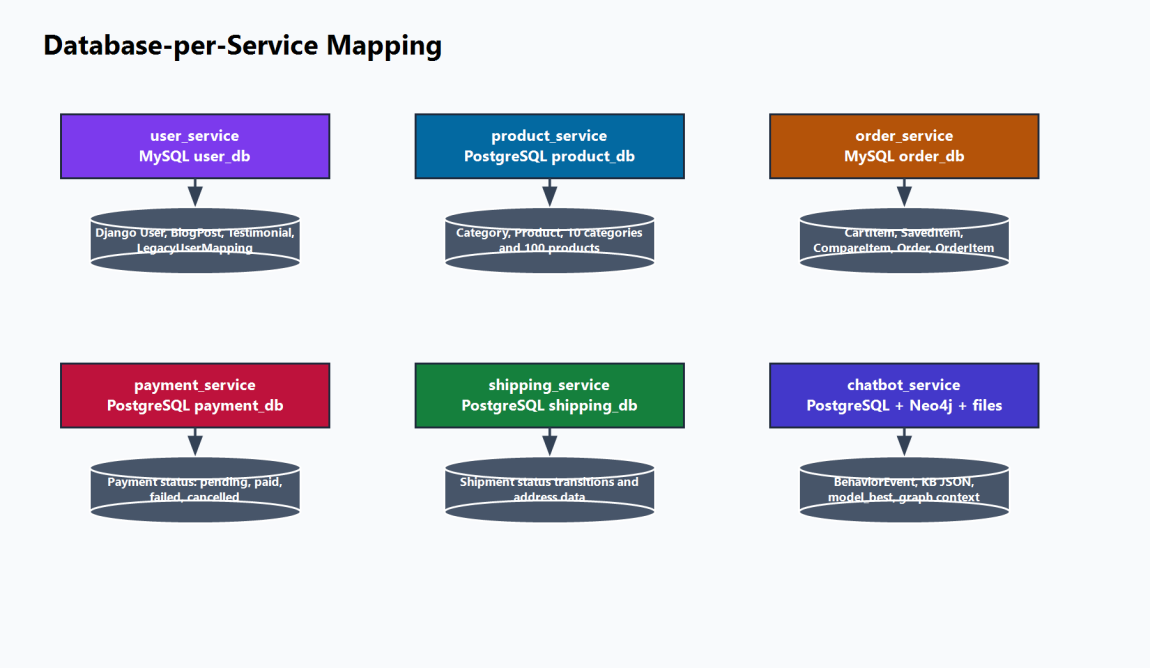


Figure 3: Database-per-service mapping in the implemented system.

6.2 Data Ownership and Duplication

Microservices often require controlled duplication. The catalog service owns products, but the order service stores product snapshots when items enter the cart and when orders are created. This prevents historical orders from changing unexpectedly if a product name, brand, image, or price changes later. Similarly, the order service stores payment and shipping status snapshots for UI compatibility, while the detailed payment and shipment records live in their own services.

The AI service also uses duplicated product context through generated knowledge-base artifacts. This is acceptable because RAG retrieval needs a local search structure and because the artifact can be rebuilt from the catalog API. The artifact is not the source of truth; `product_service` remains the source of truth for the catalog.

6.3 Database Mapping Summary

Table 3: Main database mapping by service.

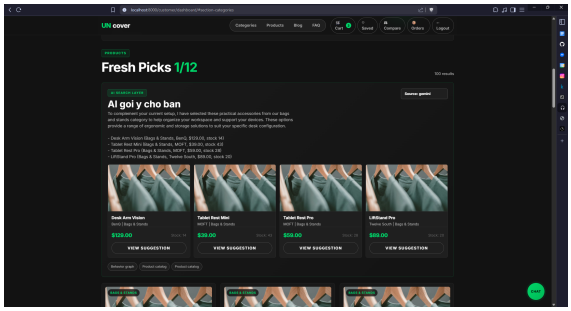
Service	Database	Main tables or artifacts
`user_service`	MySQL `user_db`	Django auth user, customer blog posts, testimonials, legacy user mappings.
`product_service`	PostgreSQL `product_db`	Category and Product.
`order_service`	MySQL `order_db`	CartItem, SavedItem, CompareItem, Order, OrderItem, OrderShipping.
`payment_service`	PostgreSQL `payment_db`	Payment record with pending, paid, failed, and cancelled states.

Service	Database	Main tables or artifacts
`shipping_service`	PostgreSQL `shipping_db`	Shipment record with pending, preparing, shipped, delivered, and cancelled states.
`chatbot_service`	PostgreSQL `chatbot_db`	BehaviorEvent records.
`chatbot_service`	Neo4j and files	User, Behavior, Category, Product graph; knowledge base JSON; model artifacts; metrics; screenshots.

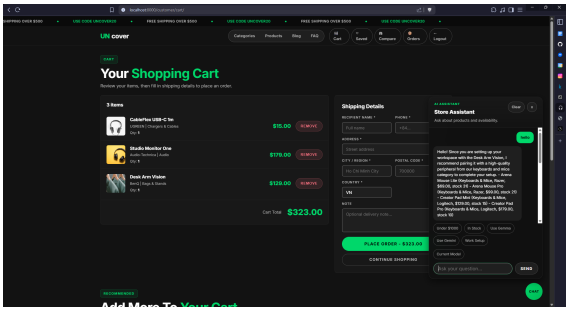
7 AI Service for Product Consultation

7.1 Purpose of the AI Service

The AI service improves the e-commerce experience by transforming passive browsing into assisted shopping. It supports three customer-facing features: dashboard suggestions based on the current search or filter context, cart suggestions for complementary products, and a floating chat widget for natural-language product consultation. These features are integrated without changing the stable customer routes; the UI calls the user service, and the user service proxies requests to the chatbot service.



(a) Dashboard AI suggestions.



(b) Cart buy-together suggestions.

Figure 4: AI recommendations integrated into customer shopping pages.

7.2 Behavior Data

The chatbot service persists behavior events in the `BehaviorEvent` model. Each event stores a `user_ref`, event type, category slug, product id, JSON metadata, and creation time. The metadata captures the message, current category, cart count, saved count, recent paid item count, device type, and price bucket. These fields allow the AI service to combine explicit search intent with historical customer context.

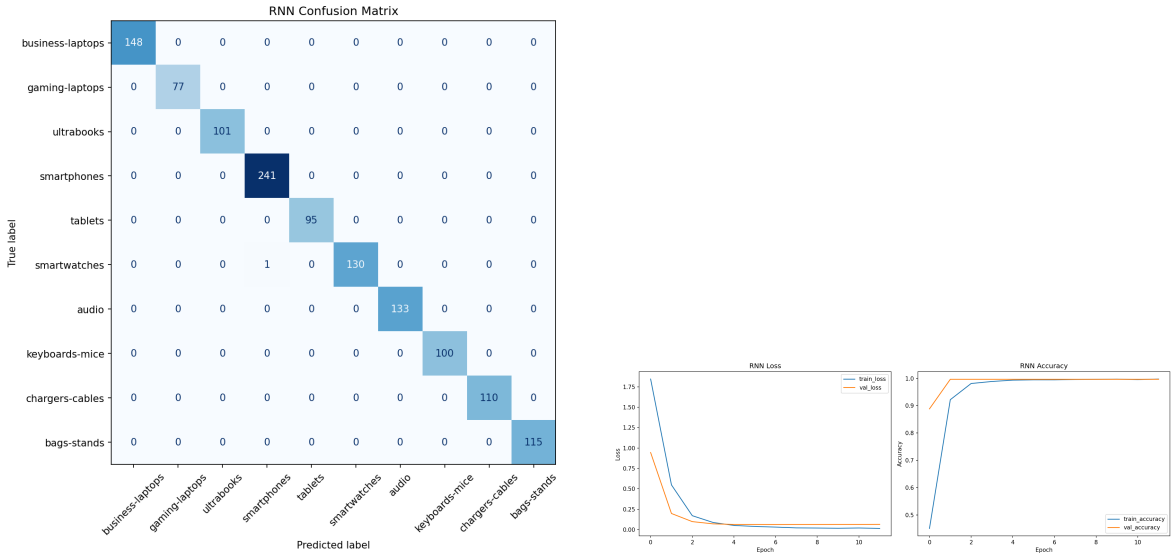
The system also contains a recovery path through `user_service`'s `backfill_chatbot_behavior` command. This command can export behavior source data from order history and send it back into the chatbot service. This is a practical design choice because behavior pipelines often fail or are introduced after transactional data already exists.

7.3 Model Selection

The artifacts include a model comparison across RNN, LSTM, and BiLSTM. All three models reach the same reported accuracy and macro F1 within a very small tolerance, but the selected `model_best` is RNN because it has the smallest parameter count and simpler inference. This selection rule is reasonable for a demo system because inference simplicity matters when the accuracy gap is negligible.

Table 4: Behavior model comparison from project artifacts.

Model	Accuracy	Macro F1	Params	Best epoch	Val. acc.	Selected
RNN	0.999201	0.999410	11192	7	0.996697	Yes
LSTM	0.999201	0.999410	29432	4	0.996697	No
BiLSTM	0.999201	0.999410	57848	2	0.997523	No



(a) Confusion matrix for the selected RNN model.

(b) Training history for the selected RNN model.

Figure 5: Model evidence used for selecting `model_best`.

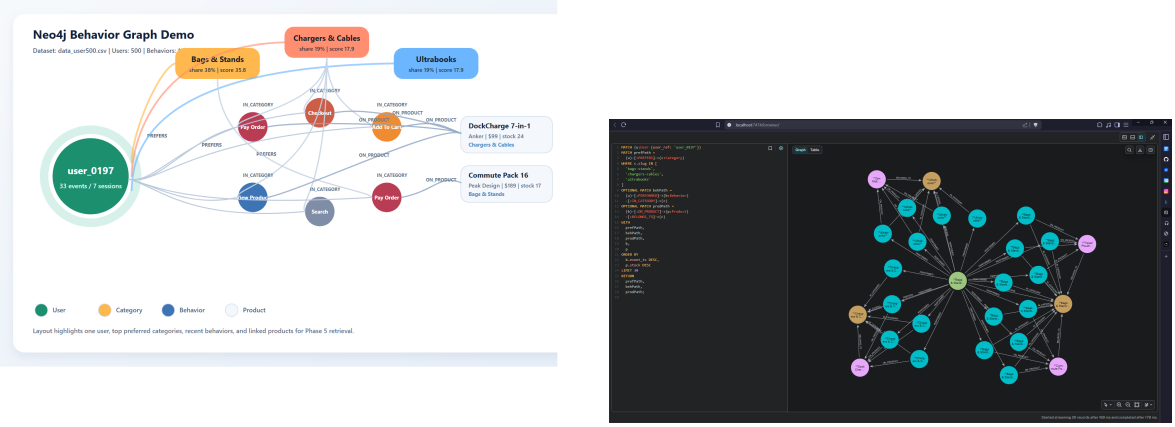
7.4 RAG Knowledge Base

The RAG layer builds a knowledge base from product data and FAQ content. The `build_chat_kb` command calls the product service, converts products into searchable documents, adds FAQ documents, tokenizes text, and writes `knowledge_base.json`. At runtime, `retrieve_rag_context` scores documents by token overlap, preferred category, stock availability, and relation to the current product. The selected documents become citations and prompt context for the LLM.

This design is intentionally lightweight. It does not require a separate vector database for Version 1, but the architecture leaves room for replacing token scoring with embeddings later. The important architectural point is that retrieval is isolated inside the chatbot service and does not leak AI-specific indexes into the catalog service.

7.5 Neo4j Behavior Graph

Neo4j is used as an optional graph knowledge base. The graph contains users, behaviors, categories, and products. Relationships such as `PERFORMED`, `IN_CATEGORY`, `ON_PRODUCT`, `BELONGS_TO`, and `PREFERS` allow the chatbot to fetch behavior context for a user or product category. If Neo4j is unavailable, the chatbot service reports the graph as unavailable and continues with file-based RAG and catalog-based recommendations.



(a) Neo4j graph evidence. (b) Neo4j query/detail evidence.

Figure 6: Behavior graph evidence for AI retrieval.

7.6 Hybrid Recommendation Pipeline

The runtime chatbot pipeline can be summarized as follows:

1. Receive a message, current product context, user context, user reference, and recommendation limit.
2. Record a behavior event in PostgreSQL.
3. Predict user behavior signal using `model_best` when available, otherwise use heuristic history and priors.
4. Detect preferred categories from the question, current product, and behavior signal.
5. Query Neo4j for graph context when available.
6. Retrieve RAG documents from the file knowledge base.
7. Rank live catalog products from `product_service`.
8. Build a prompt with user profile, dominant category, RAG documents, and recommendation candidates.
9. Call the configured LLM provider.
10. Sanitize the answer and return answer text, recommendations, citations, source, fallback status, and error code.

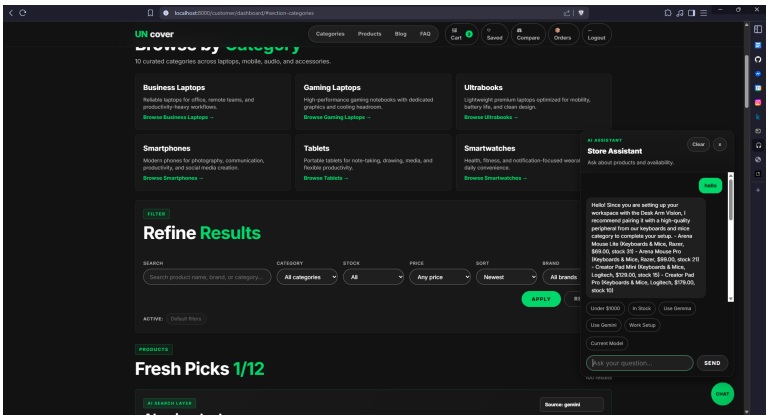


Figure 7: Customer-facing chatbot widget connected to the hybrid AI service.

8 Security and Communication Design

8.1 Authentication Surfaces

The project keeps browser authentication and API authentication separate. Customer and staff UI pages use Django sessions. JWT is used for API authentication through the auth endpoints in `user_service`. The JWT payload includes user identity, role, scopes, staff flag, and superuser flag. This coexistence is appropriate for a demo system because browser flows remain ergonomic while API flows can be tested independently.

8.2 Internal Service Authorization

Internal endpoints in `order_service` require a shared internal key passed through headers such as `X-Internal-Key`. Payment and shipping services use similar internal key concepts. This is not a complete zero-trust production security model, but it is a reasonable educational boundary because it prevents public callers from directly using internal operational endpoints without the expected key.

8.3 Fault Handling

Distributed systems fail partially. The order service handles downstream failures from payment and shipping services using `DownstreamServiceError`. The chatbot service also handles missing LLM keys, blocked keys, rate limits, network errors, unavailable Neo4j, and missing model artifacts. These fallback branches are significant because they keep the user-facing system responsive even when advanced dependencies are missing.

9 Deployment Design

9.1 Docker Compose

The system is deployed locally through Docker Compose. MySQL and PostgreSQL services have health checks. Application services depend on the relevant database health checks or service-started conditions. The chatbot service mounts the artifacts directory

so that generated knowledge bases, model files, metrics, and runtime configuration persist on the host. Neo4j exposes ports 7474 and 7687 for browser and Bolt access.

9.2 Environment-driven Configuration

Runtime URLs and credentials are controlled through environment variables. Examples include `PRODUCT_SERVICE_URL`, `ORDER_SERVICE_URL`, `PAYMENT_SERVICE_URL`, `SHIPPING_SERVICE_URL`, `CHATBOT_SERVICE_URL`, `NEO4J_URI`, `NEO4J_USERNAME`, `NEO4J_PASSWORD`, and `NEO4J_DATABASE`. This keeps service code portable across local Docker Compose and future deployment environments.

10 End-to-End Workflow

10.1 Shopping and Checkout Flow

The customer shopping flow begins at the gateway. The customer opens the dashboard, and Nginx routes the browser request to `user_service`. The user service fetches categories and products from `product_service`. When the customer adds an item to cart, the user service sends a product snapshot to `order_service`. During checkout, the order service creates the order and item snapshots, creates a payment record in `payment_service`, creates a shipment record in `shipping_service`, clears the cart, and returns the order to the user service. Payment confirmation later calls the payment service and updates the order snapshot.

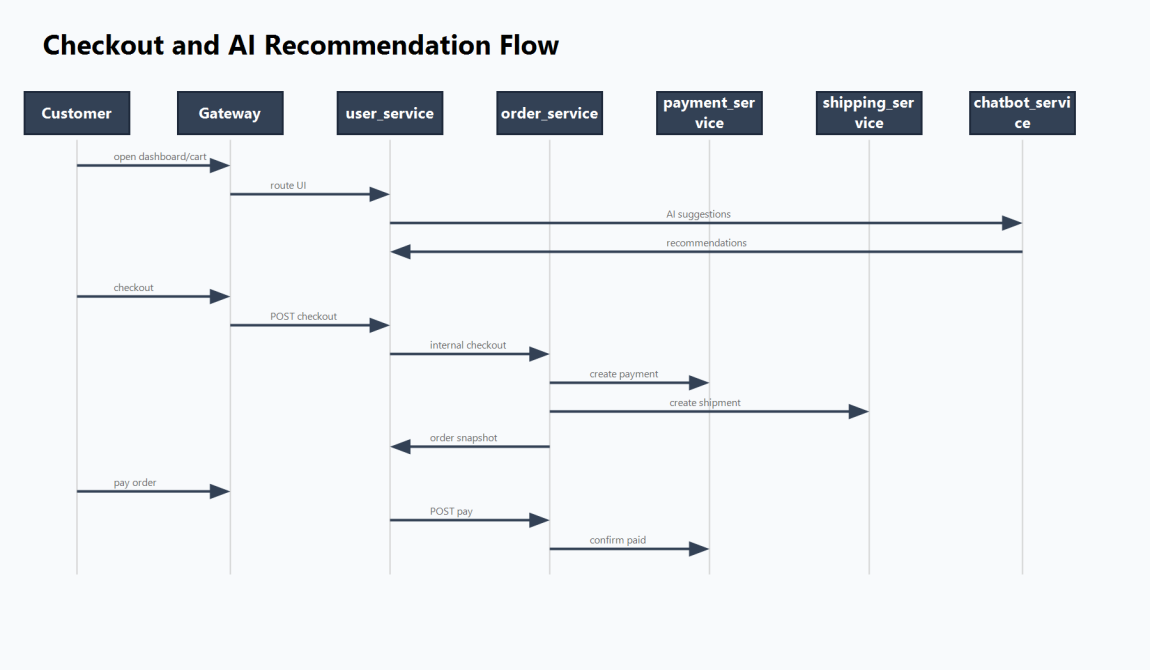


Figure 8: End-to-end checkout and AI recommendation sequence.

10.2 AI-assisted Browsing Flow

The AI flow is integrated into the same customer journey. On the dashboard, the user service builds an AI question from the current search filters and visible products. On

the cart page, it builds a buy-together question from cart item names and categories. In the chat widget, the customer provides a direct natural-language question. In all cases, the user service sends a structured payload to the chatbot service and receives answer text, recommendation cards, citations, fallback metadata, and provider metadata.

11 Evaluation and Discussion

11.1 Strengths

The most important strength of this system is boundary clarity. The six application services map to understandable business capabilities. Databases are service-local, and the order service uses snapshots instead of cross-service database joins. The Nginx gateway offers a simple public entry point while retaining direct debug ports for development. The AI service is integrated into customer workflows without forcing AI concerns into product, order, payment, or shipping services.

The second strength is graceful degradation. The chatbot can fall back from LLM calls to catalog recommendations. Neo4j is optional. Model artifacts are used when present and bypassed when unavailable. This is appropriate for educational architecture because it demonstrates that AI-enhanced systems should still remain operational when the AI layer is partially unavailable.

The third strength is migration awareness. Legacy user mapping, legacy order import, and chatbot behavior backfill show that the architecture considers recovery and cutover rather than only greenfield design.

11.2 Limitations

The main limitation is synchronous coupling during checkout. The order service directly calls payment and shipping services in the request path. This is easier to reason about for a Version 1 demo, but a production system would usually introduce asynchronous events, idempotency keys, retries, and outbox patterns to reduce failure coupling.

Another limitation is simplified security. Internal keys are useful for a local demo, but production deployment would require stronger service identity, secret management, network policies, rate limiting, audit logging, and centralized observability. Similarly, JWT validation is present for API auth, but service-to-service trust is still simplified.

The AI evaluation also has limitations. The reported model metrics are very high, which suggests the generated dataset may be clean and strongly patterned. For production use, the model should be evaluated on noisier real behavior, cold-start users, multilingual queries, out-of-stock products, and adversarial prompts. The current RAG scoring is token-based rather than embedding-based, which is acceptable for Version 1 but limited for semantic search.

11.3 Future Improvements

Future versions can improve the architecture in five directions. First, checkout should move toward event-driven orchestration using an outbox and message broker. Second, observability should be added through structured logs, distributed traces, and service-level metrics. Third, the AI service can replace token retrieval with embeddings and a vector index. Fourth, security can be strengthened with service identity, scoped tokens,

and secret rotation. Fifth, the deployment architecture can be prepared for Kubernetes by adding readiness probes, resource limits, and separate production settings.

12 Conclusion

This report analyzed a Django-based e-commerce system using microservices, DDD, Docker Compose, Nginx, database-per-service persistence, and AI-assisted shopping. The implementation demonstrates a practical transition from monolithic thinking to bounded-context thinking. The catalog, commerce, payment, shipping, identity, and AI assistant contexts have distinct ownership and communicate through explicit APIs. The order service preserves historical product snapshots, payment snapshots, and shipping snapshots for UI compatibility while delegating payment and shipping records to dedicated services.

The AI service adds a modern layer to the commerce experience. It records behavior events, uses model artifacts for category-affinity prediction, retrieves product and FAQ context through RAG, optionally queries Neo4j for graph context, and returns customer-facing recommendations through dashboard, cart, and chat interfaces. The architecture is not production-complete, but it is coherent for an academic Version 1 submission because it shows service decomposition, database ownership, gateway routing, security boundaries, deployment wiring, and AI integration in a working source-code project.

References

References

[1] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.

[2] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. O'Reilly Media, 2021.

[3] M. Fowler and J. Lewis, "Microservices: a definition of this new architectural term," 2014.

[4] C. Richardson, *Microservices Patterns: With examples in Java*. Manning Publications, 2018.

[5] Nginx, "NGINX Reverse Proxy Documentation." Official documentation.

[6] Django Software Foundation, "Django Documentation." Official documentation.

[7] Neo4j, "Neo4j Graph Database Documentation." Official documentation.

[8] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," 2020.

A Suggested Verification Commands

The following commands are suitable for local evidence collection before exporting the final PDF:

```
docker compose config --quiet
docker compose up --build -d
docker compose ps -a
docker compose logs --tail=120 gateway user_service order_service
    payment_service shipping_service product_service chatbot_service
docker compose exec product_service python manage.py seed_products --reset
docker compose exec chatbot_service python manage.py build_chat_kb --max-
    products 160
docker compose exec chatbot_service python manage.py train_behavior_model
docker compose exec chatbot_service python manage.py import_behavior_graph --
    reset
```